

# Customer State Service — Design & Implementation Plan

---

**Version:** 2.0.0  
**Last Updated:** 2026-07-29  
**Service:** Layer 1 — Behaviour Intelligence  
**Port:** 8003  
**Reference:** [docs/architecture/system-design.md](#) §8

## Table of Contents

1. [Overview & Architecture Position](#)
2. [Alignment with Reference Architecture](#)
3. [Data Model](#)
4. [State Classification Engine](#)
5. [Markov Chain Engine](#)
6. [Transition & Journey Analysis](#)
7. [API Specification](#)
8. [Repository Layer](#)
9. [Configuration](#)
10. [Integration Points](#)
11. [Implementation Roadmap](#)
12. [Testing Strategy](#)
13. [Appendix: Decision Log](#)

---

## 1. Overview & Architecture Position

### 1.1 What It Does (per [system-design.md](#) §8)

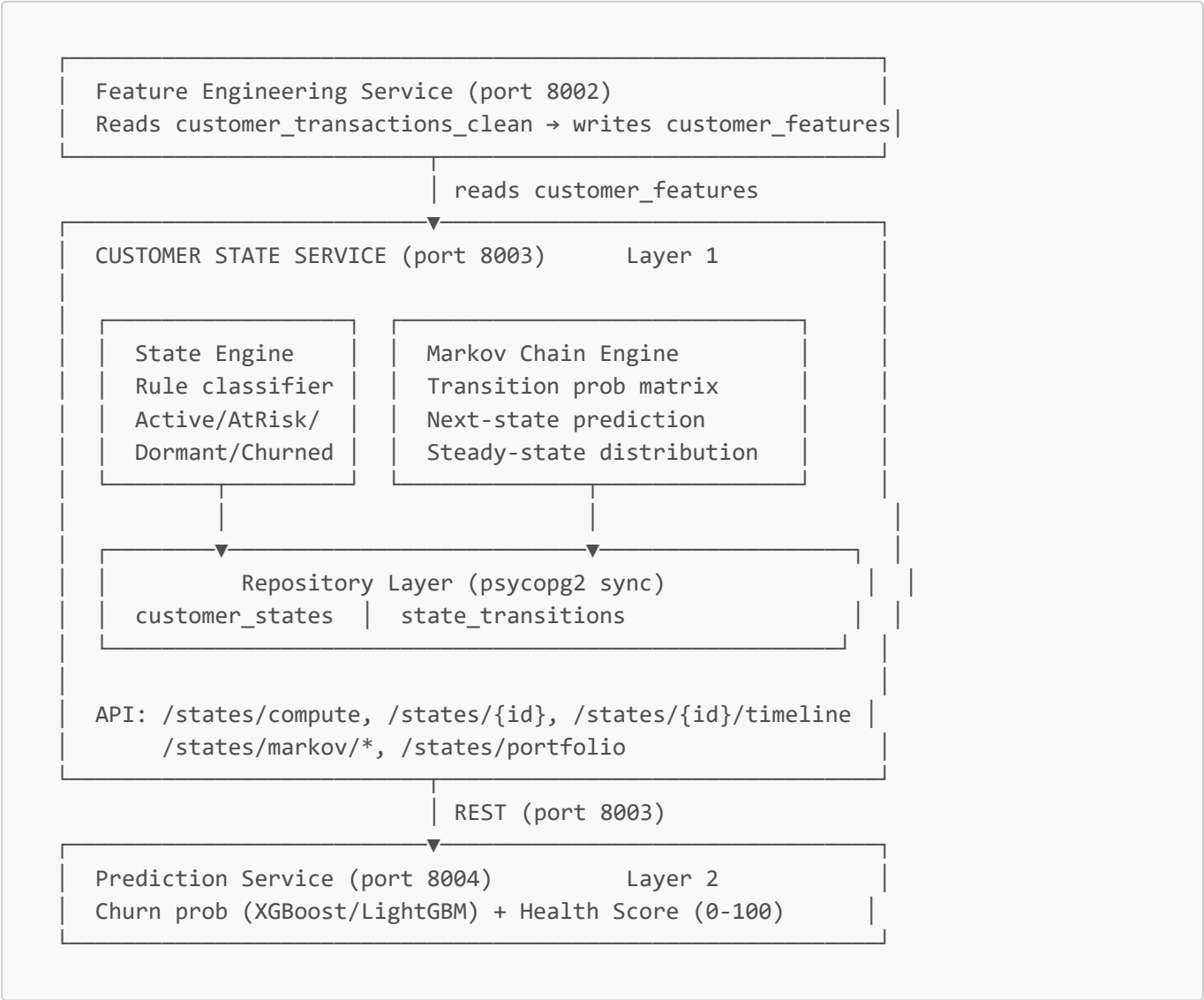
The Customer State Service is **Layer 1 of the 3-Layer AI architecture**:

```
system-design.md §8:  
  Layer 1 — Customer State Service  
  Purpose: Track customer behavioural state using Markov Chains.  
  Input:   Transaction/behaviour data from Feature Store  
  Output:  State labels: Active | At Risk | Dormant | Churned  
           Transition probabilities
```

It does NOT compute Health Scores — those belong to **Layer 2** (Prediction Service) per the reference architecture. The formula from [system-design.md](#) explicitly uses `churn_prob` from XGBoost:

$$\begin{aligned} \text{Health Score} &= (\text{churn\_weight} \times (1 - \text{churn\_prob})) && \leftarrow \text{needs Layer 2} \\ &+ (\text{clv\_weight} \times \text{clv\_percentile}) && \leftarrow \text{needs Layer 2} \\ &+ (\text{behaviour\_weight} \times \text{state\_score}) && \leftarrow \text{from Layer 1} \end{aligned}$$

1.2 Architecture Position (matching `docker-compose.yml`)



1.3 Scope Boundaries (matching ARCHITECTURE.md)

| In Scope (PoC)                            | Out of Scope (post-PoC)              |
|-------------------------------------------|--------------------------------------|
| State classification (rule-based)         | Hidden Markov Model (HMM)            |
| Markov Chain (observed transitions)       | Real-time state updates (batch only) |
| Transition matrix + next-state prediction | Push notifications / WebSocket       |
| Journey timeline + milestone detection    | Health Score (belongs to Layer 2)    |
| Portfolio aggregate queries               | Graph-based journey analysis         |

2. Alignment with Reference Architecture

This design follows three patterns established by the existing codebase.

## 2.1 Config Pattern — matching `FeatureConfig`

`services/feature-engineering-service/app/config/settings.py`:

```
class FeatureConfig(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="FE_", extra="ignore")
    engagement_recency_weight: float = 40.0
    ...

class Settings(BaseSettings):
    features: FeatureConfig = FeatureConfig()
```

→ Our equivalent (`services/customer-state-service/app/config/settings.py`):

```
class StateConfig(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="CS_", extra="ignore")
    churned_days_threshold: int = 365
    ...

class Settings(BaseSettings):
    state: StateConfig = StateConfig()
```

## 2.2 Repository Pattern — matching `FeatureRepository`

`services/feature-engineering-service/app/repository/repository.py`:

- `psycopg2` (sync) — not `asyncpg`
- `_connect()` helper with `connect_timeout`, TCP keepalives
- `psycopg2.extras.execute_values` for batch operations
- `ON CONFLICT (customer_id, as_of_date) DO UPDATE` for idempotency
- Single shared connection string: `settings.database_target_url_sync` (etl\_clean)

→ Our equivalent (`app/repository/state_repository.py`) follows this exact pattern.

## 2.3 API Pattern — matching `FeatureService`

`services/feature-engineering-service/app/api/routes.py`:

```
router = APIRouter(prefix="/features", tags=["features"])
_service = FeatureService()

@router.post("/compute-batch", response_model=ComputeBatchResponse)
def compute_batch(as_of_date: date | None = Query(default=None)) ->
    ComputeBatchResponse:
    return _service.compute_batch(as_of_date)

@router.get("/{customer_id}", response_model=FeatureSnapshot)
```

```
def get_features(customer_id: str, as_of_date: date = Query(...)) ->
FeatureSnapshot:
    result = _service.get_features(customer_id, as_of_date)
    if result is None:
        raise HTTPException(status_code=404, detail=f"No features for customer
'{customer_id}' as of {as_of_date}")
    return result
```

→ Our equivalent (`app/api/routes.py`):

```
router = APIRouter(prefix="/states", tags=["states"])
_service = StateService()

@router.post("/compute", response_model=ComputeStatesResponse)
def compute_states(as_of_date: date | None = Query(default=None)) ->
ComputeStatesResponse:
    return _service.compute_states(as_of_date)
```

## 3. Data Model

### 3.1 `customer_states` Table

```
CREATE TABLE IF NOT EXISTS public.customer_states (
    id                BIGSERIAL PRIMARY KEY,
    customer_id       VARCHAR(64) NOT NULL,
    as_of_date        DATE NOT NULL,

    -- Core classification (Layer 1)
    state             VARCHAR(16) NOT NULL
                     CHECK (state IN ('ACTIVE', 'AT_RISK', 'DORMANT', 'CHURNED')),

    -- Health Score + component scores (Layer 2 – Prediction Service)
    -- Populated by a subsequent UPDATE after Layer 2 computes churn_prob.
    -- NULL until Layer 2 runs. Layer 1 writes them as NULL.
    health_score       NUMERIC(5,2) CHECK (health_score >= 0 AND health_score <= 100),
    component_scores   JSONB DEFAULT '{}',

    -- Rule provenance (which rules fired during classification)
    classification_rules JSONB DEFAULT '{}',
    -- Example: {"active_rules": [], "risk_rules": ["30d_inactivity",
"engagement_drop"]}

    -- Metadata
    computed_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    UNIQUE (customer_id, as_of_date)
);

CREATE INDEX IF NOT EXISTS idx_customer_states_date ON customer_states(as_of_date);
```

```
CREATE INDEX IF NOT EXISTS idx_customer_states_state ON customer_states(state);
-- Partial index: only rows where Layer 2 has backfilled. Avoids wasting space
-- on NULLs (every row starts NULL until Prediction Service runs).
CREATE INDEX IF NOT EXISTS idx_customer_states_health
    ON customer_states(health_score DESC) WHERE health_score IS NOT NULL;
CREATE INDEX IF NOT EXISTS idx_customer_states_customer_date
    ON customer_states(customer_id, as_of_date DESC);
```

### 3.2 state\_transitions Table

```
CREATE TABLE IF NOT EXISTS public.state_transitions (
    id                BIGSERIAL PRIMARY KEY,
    customer_id       VARCHAR(64) NOT NULL,
    from_state        VARCHAR(16) NOT NULL,
    to_state          VARCHAR(16) NOT NULL,
    transition_date    DATE NOT NULL,

    days_in_previous_state INTEGER,
    trigger_reason     VARCHAR(256),

    -- Feature snapshot at transition time (for audit)
    feature_snapshot   JSONB DEFAULT '{}',
    -- Example: {"days_since_last_txn": 45, "engagement_score": 12.5}

    computed_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_state_transitions_customer ON state_transitions(customer_id);

-- Powers Markov transition matrix queries
CREATE INDEX idx_state_transitions_matrix
    ON state_transitions(from_state, to_state, transition_date);
```

### 3.3 DDL Location

```
database/state_engine/001_customer_states.sql
```

---

## 4. State Classification Engine

### 4.1 Rules (Ordered Precedence)

Rules are evaluated top-to-bottom; the first matching rule wins. Operators use **exclusive** upper bounds to prevent ambiguity at boundaries.

| Priority | State          | Rules                                                                                                                                  | Feature Columns Used                                                                                         |
|----------|----------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| 1        | <b>CHURNED</b> | <code>rel_customer_status = 'Closed'</code> OR<br><code>days_since_last_txn &gt; 365</code>                                            | <code>rel_customer_status</code> ,<br><code>days_since_last_txn</code>                                       |
| 2        | <b>DORMANT</b> | <code>days_since_last_txn &gt; 90</code> OR<br><code>txn_count_90d = 0</code> OR<br><code>engagement_score &lt; 10</code>              | <code>days_since_last_txn</code> ,<br><code>txn_count_90d</code> , <code>engagement_score</code>             |
| 3        | <b>AT_RISK</b> | <code>days_since_last_txn &gt;= 30</code> OR<br><code>risk_dormant_indicator = true</code> OR<br><code>engagement_score &lt; 20</code> | <code>days_since_last_txn</code> ,<br><code>risk_dormant_indicator</code> ,<br><code>engagement_score</code> |
| 4        | <b>ACTIVE</b>  | All remaining (default)                                                                                                                | —                                                                                                            |

**Boundary contract:** `days_since_last_txn = 90` is AT\_RISK (priority 3 fires before priority 2).  
`days_since_last_txn = 30` is AT\_RISK. The DORMANT rule uses strict `>` so the boundary is unambiguous regardless of evaluation order — even if someone later restructures from if/elif to a case/match, the edge case is covered by the operator, not by ordering magic.

**Health Score is NOT an input to state classification.** The `system-design.md` §8 formula uses `churn_prob` from XGBoost (Layer 2), which does not exist when Layer 1 runs. AT\_RISK triggers on observable feature-engine metrics only — `days_since_last_txn`, `risk_dormant_indicator`, and `engagement_score`. This eliminates the circularity risk entirely: State Engine runs first, produces states, then Layer 2 (Prediction Service) reads those states and computes Health Scores. No two-pass, no ordering dependency.

### Why not the alternatives?

- **Two-pass (option 2):** Requires Layer 1 → Layer 2 → Layer 1 orchestration inside a single `/states/compute` call. Violates the layer boundary and means re-running Layer 1 on a date where Layer 2 already backfilled produces different results — breaks D1's auditable-classifier guarantee.
- **Previous-date health\_score (option 3):** Means a customer who was healthy 30 days ago but crashed in the last week still has a stale `health_score` keeping them out of AT\_RISK. The lagged trigger makes the classifier non-reactive at the exact moment reactivity matters most.
- **Option 1 (chosen):** The classifier is self-contained — 100% reproducible from Feature Store columns alone. If you re-run `/states/compute` for `as_of_date=2026-07-29` six months later, you get the exact same state, regardless of whether Layer 2 has ever run.

## 4.2 Implementation

```
# services/customer-state-service/app/services/state_engine.py

class StateEngine:
    """Deterministic rule-based state classifier.

    Rules are evaluated in priority order; first match wins.
    All thresholds are configurable via StateConfig (env-prefixed: CS_).

    IMPORTANT: This classifier does NOT use health_score as an input.
    Health Score belongs to Layer 2 (Prediction Service) per system-design.md §8.
    The state engine reads only Feature Store columns – no circular dependency.
```

```

"""

def __init__(self, config: StateConfig) -> None:
    self._cfg = config

def classify(self, features: dict, previous_state: str | None = None) ->
StateResult:
    """Classify a single customer from their feature snapshot.

    Args:
        features: Dict of feature_name → value from customer_features.
            Required keys: days_since_last_txn, engagement_score,
                rel_customer_status, risk_dormant_indicator, txn_count_90d.
        previous_state: Previous state for transition-aware logic.

    Returns:
        StateResult with state and classification metadata.
        health_score and component_scores are NULL – populated by Layer 2.
    """
    ...

```

### 4.3 Configurable Thresholds

| Env Variable                    | Default | Description                                                    |
|---------------------------------|---------|----------------------------------------------------------------|
| CS_CHURNED_DAYS_THRESHOLD       | 365     | Days since last txn to mark as Churned                         |
| CS_DORMANT_DAYS_THRESHOLD       | 90      | Days since last txn (> this) to mark as Dormant                |
| CS_ATRISK_DAYS_MIN              | 30      | Days since last txn (>= this) to mark as At Risk               |
| CS_ATRISK_DAYS_MAX              | 90      | Not used directly — DORMANT's strict > handles the upper bound |
| CS_DORMANT_TXN_COUNT_THRESHOLD  | 0       | Max txn count in 90d to mark Dormant                           |
| CS_ENGAGEMENT_DORMANT_THRESHOLD | 10      | Engagement score below → Dormant                               |
| CS_ENGAGEMENT_ATRISK_MAX        | 20      | Engagement score below (strict) → At Risk                      |

Note: `CS_HEALTH_ATRISK_THRESHOLD` is intentionally **absent**. Health Score belongs to Layer 2 (Prediction Service) per `system-design.md` §8. Layer 1 classifies purely from Feature Store observables.

### 4.4 StateResult Schema

```

class StateResult(BaseModel):
    customer_id: str
    as_of_date: date
    state: Literal["ACTIVE", "AT_RISK", "DORMANT", "CHURNED"]
    classification_rules: dict # {"risk_rules": ["30d_inactivity"], "active_rules":
[], ...}
    previous_state: str | None
    is_transition: bool # True if state changed from previous
    # health_score and component_scores are NOT in StateResult.

```

```
# They are populated separately by Layer 2 (Prediction Service)
# and stored in customer_states via a subsequent UPDATE.
```

## 5. Markov Chain Engine

### 5.1 Purpose

Computes the 4×4 state transition probability matrix from observed transitions — matching [system-design.md](#) §8 Layer 1 output: "Transition probabilities".

Used to:

- Predict next-state probabilities for a given customer
- Identify high-risk transition paths (Active → At Risk → Dormant → Churned)
- Compute steady-state distribution of the customer portfolio

### 5.2 Transition Matrix (Example)

|      |         | TO     |        |         |         |                   |
|------|---------|--------|--------|---------|---------|-------------------|
|      |         | Active | AtRisk | Dormant | Churned |                   |
| FROM | Active  | 0.85   | 0.10   | 0.03    | 0.02    |                   |
|      | AtRisk  | 0.15   | 0.50   | 0.25    | 0.10    |                   |
|      | Dormant | 0.05   | 0.10   | 0.60    | 0.25    |                   |
|      | Churned | 0.00   | 0.00   | 0.00    | 1.00    | ← absorbing state |

### 5.3 SQL Query for Matrix Data

```
-- Aggregate transition counts for matrix
SELECT
    from_state,
    to_state,
    COUNT(*) AS transition_count
FROM state_transitions
WHERE transition_date > ((d)s::date - INTERVAL '%(window_days)s days')
    AND transition_date <= (d)s::date
GROUP BY from_state, to_state;
```

### 5.4 Implementation

```
# services/customer-state-service/app/engines/markov/

class MarkovEngine:
    """First-order discrete-time Markov Chain for customer states.

    Builds transition probability matrix from observed state_transitions.
    Predicts next-state probabilities and steady-state distribution.
```

```

Dependencies: numpy (matrix ops, eigenvector) – already in requirements.txt.
"""

def __init__(self) -> None:
    self._matrix: np.ndarray | None = None      # 4x4 probability matrix
    self._state_index: dict[str, int] = {}      # state_name → matrix row/col

def fit(self, transitions: list[TransitionRecord]) -> None:
    """Build transition matrix from observed transitions."""
    ...

def predict_next(self, current_state: str) -> dict[str, float]:
    """Return {next_state: probability} for a given current state."""
    ...

def steady_state(self) -> dict[str, float]:
    """Compute steady-state distribution (eigenvector of eigenvalue 1)."""
    ...

@property
def matrix(self) -> list[list[float]]:
    """Return the 4x4 probability matrix as nested lists."""
    ...

```

## 5.5 Cold-Start & Edge-Case Handling

`CS_MARKOV_MIN_TRANSITIONS` (default 50) governs when a matrix row is considered reliable.

### `/states/markov/matrix` response contract:

When a `from_state` has fewer than `min_transitions` observed transitions in the window, the API response must signal this rather than silently returning a noisy probability row. The contract:

```
{
  "matrix": [
    [0.85, 0.10, 0.03, 0.02],
    [0.15, 0.50, 0.25, 0.10],
    [null, null, null, null],
    [0.00, 0.00, 0.00, 1.00]
  ],
  "warnings": [
    {
      "state": "DORMANT",
      "observed_transitions": 23,
      "min_required": 50,
      "action": "row_masked"
    }
  ],
  "window_days": 180,
  "total_transitions_observed": 1234
}
```

- A `null` row means "insufficient data" — callers must handle this.
- A `warnings` array signals which rows were masked and why.
- CHURNED is a special case: if zero transitions are observed from CHURNED (shouldn't happen since it's absorbing), the row defaults to `[0, 0, 0, 1]` — the absorbing-state prior.

#### `/states/markov/predict/{customer_id}` response contract:

When the current state has insufficient observations:

```
{
  "customer_id": "CUST00001",
  "current_state": "DORMANT",
  "predictions": null,
  "warning": "Insufficient transitions from DORMANT (23 observed, need 50). Try a wider window or re-run after more data accumulates."
}
```

#### `steady_state()` degenerate handling:

`compute_steady_state()` returns `None` if the transition matrix has any zero-row (no outgoing transitions from a non-absorbing state). The API endpoint maps this to a 503 with a clear message. In practice this shouldn't occur given the 4-state design with CHURNED absorbing, but the guard prevents a cryptic `LinAlgError` if a new state type is added later.

## 5.6 Data Freshness Contract — Batch-Only Reads

All single-customer reads (`GET /states/{customer_id}`, `/states/{customer_id}/timeline`) return rows that were **already computed by a prior POST `/states/compute` batch run**. There is no on-demand classification path.

This means:

- `GET /states/{customer_id}?as_of_date=2026-07-29` returns a 404 if `/states/compute` hasn't run for that date yet.
- The `computed_at` timestamp on each `customer_states` row is the moment the batch finished — not the request time.
- If a user needs state for today, the expected flow is: `POST /states/compute` → `GET /states/{id}` (synchronous within the same session) or rely on a scheduled batch (e.g., nightly).

This contract eliminates the need for the state engine to run on-demand, which keeps `classify()` a pure batch function and avoids the CLV-percentile-per-request problem entirely (percentiles are pre-computed during the batch run and stored in `component_scores`).

## 6.1 Transition Analyzer

Detects state changes between two consecutive `as_of_date` snapshots and attributes a trigger reason. Follows the generator pattern from `FeaturePipeline` — each stage commits independently, idempotent by design.

| Transition       | Trigger Reasons (checked in order)                                      |
|------------------|-------------------------------------------------------------------------|
| Any → CHURNED    | Account closed, 365d inactivity                                         |
| Any → DORMANT    | 90d inactivity, zero txns in 90d, engagement collapse                   |
| Any → AT_RISK    | 30d inactivity, dormant indicator, engagement decay, balance drop > 50% |
| AT_RISK → ACTIVE | Activity resumed, balance recovery                                      |
| DORMANT → ACTIVE | Re-engagement (first txn after dormancy)                                |

```
# services/customer-state-service/app/services/transition_analyzer.py

class TransitionAnalyzer:
    """Detects state changes and attributes trigger reasons."""

    def detect(
        self,
        current: StateResult,
        previous: StateResult | None,
        feature_delta: dict | None = None,
    ) -> TransitionRecord | None:
        """Return TransitionRecord if state changed, None if unchanged."""
        ...
```

## 6.2 Journey Analyzer

Higher-level journey analysis: milestone detection, average time-in-state, most common state sequences.

| Milestone                      | Detection Rule                                                            |
|--------------------------------|---------------------------------------------------------------------------|
| <code>first_transaction</code> | First row in <code>customer_transactions_clean</code>                     |
| <code>first_loan</code>        | First row in <code>loans_clean</code> with <code>status = 'ACTIVE'</code> |
| <code>first_churn_risk</code>  | First <code>state = 'AT_RISK'</code> in <code>customer_states</code>      |

| Milestone      | Detection Rule                             |
|----------------|--------------------------------------------|
| first_dormancy | First state = 'DORMANT' in customer_states |
| recovery       | Transition from AT_RISK/DORMANT → ACTIVE   |

```
# services/customer-state-service/app/services/journey_analyzer.py

class JourneyAnalyzer:
    """Analyzes customer lifecycle journeys and detects milestones."""

    def detect_milestones(self, customer_id: str) -> list[Milestone]:
        ...

    def avg_time_in_state(self, as_of_date: date) -> dict[str, float]:
        ...

    def top_paths(self, n: int = 5) -> list[JourneyPath]:
        ...
```

## 7. API Specification

### 7.1 Endpoints (matching `FeatureService` router pattern)

| Method | Endpoint                             | Purpose                       | Matches                        |
|--------|--------------------------------------|-------------------------------|--------------------------------|
| POST   | /states/compute                      | Classify all customers        | /features/compute-batch        |
| GET    | /states/{customer_id}                | Get state snapshot            | /features/{customer_id}        |
| GET    | /states/{customer_id}/timeline       | Full state history            | /features/{customer_id}/latest |
| GET    | /states/portfolio                    | Aggregate by state/branch     | —                              |
| GET    | /states/markov/matrix                | Transition probability matrix | —                              |
| GET    | /states/markov/predict/{customer_id} | Next-state probabilities      | —                              |
| GET    | /health                              | Service health check          | /health (standard)             |

Note: There is **no** /states/{id}/health endpoint. Health Scores belong to Layer 2 (Prediction Service) per [system-design.md §8](#).

## 7.2 Response Schemas

### GET /states/{customer\_id}

**Response immediately after /states/compute (Layer 1 only — Layer 2 has not run yet):**

```
{
  "customer_id": "CUST00001",
  "as_of_date": "2026-07-29",
  "state": "AT_RISK",
  "previous_state": "ACTIVE",
  "is_transition": true,
  "classification_rules": {
    "risk_rules": ["30d_inactivity", "engagement_drop"]
  },
  "health_score": null,
  "component_scores": {},
  "computed_at": "2026-07-29T14:30:00Z"
}
```

**Response after Layer 2 backfills (Prediction Service has run):**

```
{
  "customer_id": "CUST00001",
  "as_of_date": "2026-07-29",
  "state": "AT_RISK",
  "previous_state": "ACTIVE",
  "is_transition": true,
  "classification_rules": {
    "risk_rules": ["30d_inactivity", "engagement_drop"]
  },
  "health_score": 42.5,
  "component_scores": {
    "churn_risk_sub": 60.0,
    "clv_percentile_sub": 35.0,
    "behaviour_sub": 32.5
  },
  "computed_at": "2026-07-29T14:30:00Z"
}
```

**Frontend contract:** `health_score: null` is a **normal, expected state**, not an error. The Customer Detail screen should render a "Health Score pending" placeholder (greyed-out gauge, "Awaiting scoring" label) until Layer 2 has run. `component_scores: {}` means the same thing — no sub-scores available yet.

### GET /states/markov/matrix

```
{
  "as_of_date": "2026-07-29",
  "window_days": 180,
```

```

"states": ["ACTIVE", "AT_RISK", "DORMANT", "CHURNED"],
"matrix": [
  [0.85, 0.10, 0.03, 0.02],
  [0.15, 0.50, 0.25, 0.10],
  [0.05, 0.10, 0.60, 0.25],
  [0.00, 0.00, 0.00, 1.00]
],
"steady_state": {
  "ACTIVE": 0.45, "AT_RISK": 0.20,
  "DORMANT": 0.25, "CHURNED": 0.10
}
}

```

#### GET /states/{customer\_id}/timeline

```

{
  "customer_id": "CUST00001",
  "timeline": [
    {"as_of_date": "2026-01-15", "state": "ACTIVE"},
    {"as_of_date": "2026-07-15", "state": "AT_RISK"}
  ],
  "transitions": [
    {
      "transition_date": "2026-07-15",
      "from_state": "ACTIVE",
      "to_state": "AT_RISK",
      "trigger_reason": "30d inactivity threshold crossed (days_since_last_txn=45)",
      "days_in_previous_state": 181
    }
  ]
}

```

#### GET /states/portfolio

```

{
  "as_of_date": "2026-07-29",
  "branch_code": "BR001",
  "total_customers": 12450,
  "by_state": {
    "ACTIVE": {"count": 8500, "pct": 68.3},
    "AT_RISK": {"count": 2200, "pct": 17.7},
    "DORMANT": {"count": 1200, "pct": 9.6},
    "CHURNED": {"count": 550, "pct": 4.4}
  }
}

```

## 8. Repository Layer

## 8.1 Pattern Reference

Mirrors `services/feature-engineering-service/app/repository/repository.py` exactly:

| FeatureRepository Pattern                                                    | → StateRepository Equivalent        |
|------------------------------------------------------------------------------|-------------------------------------|
| <code>settings.database_target_url_sync</code>                               | Same — reads <code>etl_clean</code> |
| <code>psycopg2.connect(dsn, connect_timeout=10, keepalives=...)</code>       | Same <code>_connect()</code> helper |
| <code>psycopg2.extras.execute_values(cur, sql, values, page_size=...)</code> | Same batch upsert                   |
| <code>ON CONFLICT (customer_id, as_of_date) DO UPDATE</code>                 | Same idempotency                    |
| <code>cur.execute(sql, params) with %(as_of_date)s::date</code>              | Same parameterized SQL              |
| <code>conn.commit()</code> after each batch                                  | Same per-stage commit               |

## 8.2 StateRepository

```
# services/customer-state-service/app/repository/state_repository.py

class StateRepository:
    """Data access for customer_states table.

    Mirrors FeatureRepository: psycopg2 sync, _connect() with timeouts,
    batch upsert via execute_values, ON CONFLICT for idempotency.
    """

    def __init__(self) -> None:
        self._conn_str = settings.database_target_url_sync # etl_clean

    def _connect(self) -> psycopg2.extensions.connection:
        """Mirrors FeatureRepository._connect() - timeouts + keepalives."""
        return psycopg2.connect(
            self._conn_str,
            connect_timeout=10,
            keepalives=1,
            keepalives_idle=30,
            keepalives_interval=10,
            keepalives_count=3,
        )

    def upsert_batch(self, states: list[dict], as_of_date: date) -> int:
        """Batch upsert. Idempotent via ON CONFLICT (customer_id, as_of_date)."""
        ...

    def get_state(self, customer_id: str, as_of_date: date) -> dict | None:
        ...

    def get_timeline(self, customer_id: str, limit: int = 50) -> list[dict]:
        ...
```

```
def get_portfolio_summary(self, as_of_date: date, branch_code: str | None =
None) -> dict:
    ...
```

## 8.3 JourneyRepository

```
# services/customer-state-service/app/repository/journey_repository.py

class JourneyRepository:
    """Data access for state_transitions table. Same pattern as StateRepository."""

    def __init__(self) -> None:
        self._conn_str = settings.database_target_url_sync

    def insert_transitions(self, transitions: list[dict]) -> int:
        ...

    def get_transitions(self, customer_id: str) -> list[dict]:
        ...

    def get_transition_matrix_data(
        self, as_of_date: date, window_days: int = 180
    ) -> list[dict]:
        """Aggregated (from_state, to_state, count) for Markov matrix."""
        ...
```

## 9. Configuration

### 9.1 Pattern Reference

Mirrors `services/feature-engineering-service/app/config/settings.py`:

```
# Reference: FeatureConfig (feature-engineering-service)
class FeatureConfig(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="FE_", extra="ignore")
    engagement_recency_weight: float = 40.0
    ...

class Settings(BaseSettings):
    features: FeatureConfig = FeatureConfig()
```

→ Our equivalent:

### 9.2 StateConfig

```
# services/customer-state-service/app/config/settings.py

class StateConfig(BaseSettings):
    """State classification thresholds. Env-prefixed CS_. Mirrors FeatureConfig
    pattern."""

    model_config = SettingsConfigDict(env_prefix="CS_", extra="ignore")

    # ---- State classification thresholds ----
    churned_days_threshold: int = 365
    dormant_days_threshold: int = 90
    atrisk_days_min: int = 30
    atrisk_days_max: int = 90
    dormant_txn_count_threshold: int = 0
    engagement_dormant_threshold: float = 10.0
    engagement_atrisk_max: float = 20.0

    # ---- Markov Chain ----
    markov_window_days: int = 180
    markov_min_transitions: int = 50

    # ---- Journey Analysis ----
    journey_top_paths: int = 5

class Settings(BaseSettings):
    """Service settings. Mirrors Feature Engineering Settings pattern."""
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8",
    extra="ignore")
    environment: str = "development"
    log_level: str = "INFO"
    service_port: int = 8003
    state: StateConfig = StateConfig()
```

### 9.3 main.py Pattern

```
# services/customer-state-service/main.py
# Mirrors feature-engineering-service/main.py exactly

import os, sys
_project_root =
os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
if _project_root not in sys.path:
    sys.path.insert(0, _project_root)

from dotenv import load_dotenv
load_dotenv(os.path.join(_project_root, ".env"))

from fastapi import FastAPI
from app.api.routes import router
```

```
app = FastAPI(title="Customer State Service", version="1.0.0")
app.include_router(router)

@app.get("/health")
def health() -> dict:
    return {"status": "healthy", "service": "customer-state-service"}
```

## 10. Integration Points

### 10.1 Upstream — Data Sources (matching `docker-compose.yml` service graph)

| Service                    | DB/Table                                           | Purpose                                                            |
|----------------------------|----------------------------------------------------|--------------------------------------------------------------------|
| Feature Engineering (8002) | <code>etl_clean.customer_features</code>           | Feature snapshot per customer per date — input to state classifier |
| ETL Engine                 | <code>etl_clean.customers_clean</code>             | <code>rel_customer_status</code> for CHURNED detection             |
| ETL Engine                 | <code>etl_clean.accounts_clean</code>              | Product types for relationship context                             |
| ETL Engine                 | <code>etl_clean.customer_transactions_clean</code> | Milestone detection (first_txn)                                    |

### 10.2 Downstream — Consumers

| Service                      | What It Reads                                | Purpose                                               |
|------------------------------|----------------------------------------------|-------------------------------------------------------|
| API Gateway (8080)           | <code>GET /states/{id}</code>                | Customer Detail → State Timeline (FR-CUST-03)         |
| API Gateway (8080)           | <code>GET /states/portfolio</code>           | RM Dashboard → Portfolio summary (FR-DASH-01)         |
| Prediction Service (8004)    | <code>etl_clean.customer_states</code>       | State as input feature for churn prediction           |
| Prediction Service (8004)    | <code>GET /states/markov/predict/{id}</code> | Next-state probabilities → Layer 2 Health Score input |
| Decision Intelligence (8005) | <code>etl_clean.customer_states</code>       | State-based NBA filtering                             |

### 10.3 Gateway Route Registration

Mirroring existing `gateway/app/routes/` patterns — simple HTTP proxy:

```
# gateway/app/routes/customer_state.py
# Proxy: /api/states/* → http://customer-state-service:8003/states/*
```

## 11. Implementation Roadmap

### Phase 1 — Data Foundation

| Task                                     | File                                                       | Refs                                       |
|------------------------------------------|------------------------------------------------------------|--------------------------------------------|
| Create DDL                               | <code>database/state_engine/001_customer_states.sql</code> | §3                                         |
| Implement <code>StateConfig</code>       | <code>app/config/settings.py</code>                        | §9, mirrors <code>FeatureConfig</code>     |
| Implement Pydantic schemas               | <code>app/schemas/state.py</code>                          | §4.4 <code>StateResult</code>              |
| Implement Pydantic schemas               | <code>app/schemas/transition.py</code>                     | <code>TransitionRecord</code>              |
| Implement <code>StateRepository</code>   | <code>app/repository/state_repository.py</code>            | §8, mirrors <code>FeatureRepository</code> |
| Implement <code>JourneyRepository</code> | <code>app/repository/journey_repository.py</code>          | §8                                         |

### Phase 2 — Business Logic

| Task                                                     | File                                             | Refs |
|----------------------------------------------------------|--------------------------------------------------|------|
| Implement <code>StateEngine.classify()</code>            | <code>app/services/state_engine.py</code>        | §4   |
| Implement <code>TransitionAnalyzer.detect()</code>       | <code>app/services/transition_analyzer.py</code> | §6.1 |
| Implement <code>JourneyAnalyzer</code>                   | <code>app/services/journey_analyzer.py</code>    | §6.2 |
| Implement <code>MarkovEngine</code> (fit/predict/steady) | <code>app/engines/markov/</code>                 | §5   |

### Phase 3 — API Layer

| Task                                                       | File                                       | Refs                                                   |
|------------------------------------------------------------|--------------------------------------------|--------------------------------------------------------|
| Implement <code>StateService</code> (business logic entry) | <code>app/services/state_service.py</code> | mirrors <code>FeatureService</code>                    |
| Implement 7 endpoints                                      | <code>app/api/routes.py</code>             | §7, mirrors <code>features/routes.py</code>            |
| Implement DI dependencies                                  | <code>app/api/dependencies.py</code>       |                                                        |
| Wire <code>main.py</code> with lifespan                    | <code>main.py</code>                       | §9.3, mirrors <code>feature-engineering/main.py</code> |

### Phase 4 — Integration

| Task                           | File                                              |
|--------------------------------|---------------------------------------------------|
| Register routes in Gateway     | <code>gateway/app/routes/customer_state.py</code> |
| Seed initial state computation | Create <code>scripts/seed_states.py</code>        |

| Task             | File                                             |
|------------------|--------------------------------------------------|
| Integration test | <code>tests/integration/test_state_api.py</code> |

## Phase 5 — Testing

| Task                         | File                                                 |
|------------------------------|------------------------------------------------------|
| Unit: state engine rules     | <code>tests/unit/test_state_engine.py</code>         |
| Unit: Markov engine          | <code>tests/unit/test_markov.py</code>               |
| Integration: compute → query | <code>tests/integration/test_state_api.py</code>     |
| E2E: Gateway → State → DB    | <code>tests/integration/test_gateway_proxy.py</code> |

## 12. Testing Strategy

### 12.1 Unit Tests (matching `test_validation_ground_truth.py` style)

```
# tests/unit/test_state_engine.py

def test_active_customer_recent_activity():
    """days_since_last_txn=5, engagement=75 → ACTIVE."""
    engine = StateEngine(StateConfig())
    assert engine.classify({"days_since_last_txn": 5, "engagement_score": 75}).state == "ACTIVE"

def test_at_risk_30d_inactive():
    """days_since_last_txn=45 → AT_RISK."""
    assert engine.classify({"days_since_last_txn": 45, "engagement_score": 50}).state == "AT_RISK"

def test_dormant_90d_inactive():
    """days_since_last_txn=120 → DORMANT."""
    assert engine.classify({"days_since_last_txn": 120, "engagement_score": 5}).state == "DORMANT"

def test_churned_closed_account():
    """rel_customer_status='Closed' → CHURNED (priority 1)."""
    assert engine.classify({"rel_customer_status": "Closed", "days_since_last_txn": 5}).state == "CHURNED"

def test_churned_beats_dormant():
    """CHURNED (prio 1) takes precedence over DORMANT (prio 2)."""
    assert engine.classify({"rel_customer_status": "Closed", "days_since_last_txn": 120}).state == "CHURNED"

def test_boundary_90_days_is_at_risk():
    """At exactly 90 days, AT_RISK fires (prio 3 > prio 2 because DORMANT uses strict >)."""
    features = {"days_since_last_txn": 90, "engagement_score": 15, "txn_count_90d": 3}
```

```

assert engine.classify(features).state == "AT_RISK"
# Verify it is NOT DORMANT – DORMANT requires > 90 (strict)
assert engine.classify(features).state != "DORMANT"

def test_boundary_30_days_is_at_risk():
    """At exactly 30 days, AT_RISK fires (>= 30)."""
    features = {"days_since_last_txn": 30, "engagement_score": 25}
    assert engine.classify(features).state == "AT_RISK"

def test_boundary_29_days_is_active():
    """At 29 days, neither AT_RISK nor DORMANT fire → ACTIVE (default)."""
    features = {"days_since_last_txn": 29, "engagement_score": 25}
    assert engine.classify(features).state == "ACTIVE"

def test_boundary_91_days_is_dormant():
    """At 91 days, DORMANT fires (> 90, strict)."""
    features = {"days_since_last_txn": 91, "engagement_score": 15}
    assert engine.classify(features).state == "DORMANT"

```

## 12.2 Markov Tests

```

# tests/unit/test_markov.py

def test_matrix_rows_sum_to_one():
    """Each row in the Markov matrix must sum to 1.0."""

def test_churn_is_absorbing():
    """CHURNED → CHURNED probability must be 1.0."""

def test_steady_state_satisfies_pi_p_equals_pi():
    """ $\pi P = \pi$  for steady-state distribution."""

```

## 13. Appendix: Decision Log

| #  | Decision                                 | Rationale                                                                                                                   | Ref                                                 |
|----|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| D1 | Rule-based state classification (not ML) | Transparent, auditable, zero training data. PoC-appropriate.                                                                | ARCHITECTURE.md §PoC scope                          |
| D2 | Health Score NOT in Layer 1              | <a href="#">system-design.md</a> §8 defines it in Layer 2 (Prediction Service) with <a href="#">churn_prob</a> from XGBoost | <a href="#">system-design.md</a> §8                 |
| D3 | HMM deferred to post-PoC                 | <a href="#">ARCHITECTURE.md</a> explicitly scopes HMM out of <a href="#">poc-90day</a>                                      | <a href="#">ARCHITECTURE.md</a>                     |
| D4 | psycpg2 (sync) — not asynpg              | Same pattern as <a href="#">FeatureRepository</a> . Faster for batch ops.                                                   | <a href="#">feature-engineering/app/repository/</a> |

| #   | Decision                                                                  | Rationale                                                                                                                                                                        | Ref                                    |
|-----|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| D5  | <code>customer_states</code> in <code>etl_clean</code>                    | Co-located with <code>customer_features</code> — reduces cross-DB joins                                                                                                          | <code>docker-compose.yml</code>        |
| D6  | Port 8003 (Layer 1 in 8002→8003→8004 sequence)                            | Matches <code>docker-compose.yml</code> service graph                                                                                                                            | <code>docker-compose.yml</code>        |
| D7  | <code>component_scores</code> JSONB (if needed)                           | Allows adding columns without DDL migration                                                                                                                                      | Feature Store pattern                  |
| D8  | <code>ON CONFLICT DO UPDATE</code>                                        | Idempotent — matches <code>customer_features</code> upsert pattern                                                                                                               | feature-engineering/repository         |
| D9  | Config pattern: <code>StateConfig</code> with <code>env_prefix CS_</code> | Mirrors <code>FeatureConfig</code> with <code>FE_</code> prefix                                                                                                                  | feature-engineering/config/settings.py |
| D10 | API pattern: <code>APIRouter</code> + <code>StateService</code>           | Mirrors <code>APIRouter</code> + <code>FeatureService</code>                                                                                                                     | feature-engineering/api/routes.py      |
| D11 | Strict <code>&gt;</code> for DORMANT (not <code>&gt;=</code> )            | <code>days_since_last_txn = 90</code> is AT_RISK. Boundary is operator-enforced — safe if someone restructures to case/match.                                                    | §4.1                                   |
| D12 | Markov cold-start: null rows + warnings                                   | Silently returning noisy probabilities from 5 observations is worse than an explicit null. Callers must handle null rows.                                                        | §5.5                                   |
| D13 | No on-demand state classification                                         | All single-customer reads come from pre-computed batch rows. 404 if batch hasn't run for that date.                                                                              | §5.6                                   |
| D14 | Absorbing-state prior for CHURNED                                         | Zero observed CHURNED transitions → default to <code>[0,0,0,1]</code> . Prevents degenerate matrix.                                                                              | §5.5                                   |
| D15 | <code>health_score</code> excluded from state classification              | Two-pass breaks auditability (D1). Previous-date creates stale triggers. Self-contained Feature Store-only classifier is 100% reproducible regardless of Layer 2 backfill state. | §4.1                                   |
| D16 | Partial index on <code>health_score</code>                                | <code>WHERE health_score IS NOT NULL</code> — avoids indexing 100% NULL columns before Layer 2 runs. Index stays lean.                                                           | §3.1                                   |
| D17 | <code>health_score: null</code> is a valid API response                   | Frontend must render "pending" placeholder. Documented as a normal expected state, not an error, in the API contract.                                                            | §7.2                                   |

| #   | Decision                                                                                                       | Rationale                                                                                                                                                                                                                                                                                                                                             | Ref   |
|-----|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| D18 | <code>days &gt; 90</code> $\Rightarrow$<br><code>txn_count_90d = 0</code> is a derived invariant, not enforced | The DORMANT rule has both <code>days &gt; 90</code> and <code>txn_count_90d = 0</code> as OR branches. Currently always redundant, but if Feature Engine ever decouples them (e.g., window boundary change, counting pending txns), the <code>days &gt; 90</code> branch is the safety net. Pinned by <code>test_dormant_days_gt_90_fallback</code> . | \$4.1 |
| D19 | Engagement collapse $\rightarrow$ DORMANT (not AT_RISK) is intentional                                         | A customer still transacting (auto-debits, minimum required) but with <code>engagement_score &lt; 10</code> is functionally dormant — near-zero active engagement. Skipping AT_RISK is correct: the signal is severity, not progression. Pinned by <code>test_dormant_engagement_only</code> .                                                        | \$4.1 |